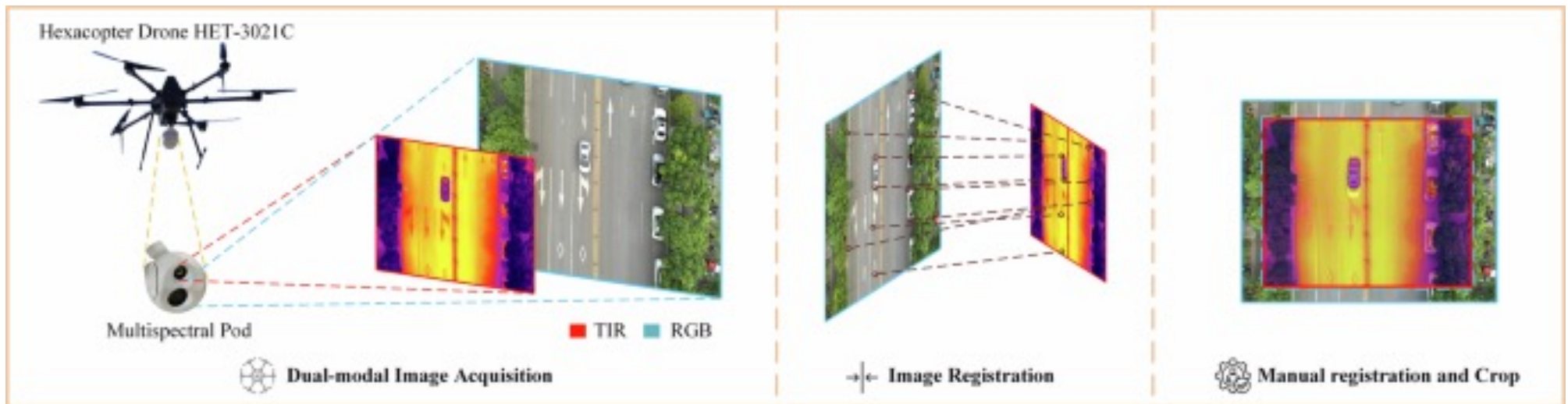


Registrazione di Immagini

Image Registration: Feature-based & Multimodal

Obiettivo

- allineare immagini
- tracking, SLAM, medical imaging, immagini aeree



Cos'è l'Image Registration

- trovare (automaticamente) trasformazione T che allinea due immagini
- Può essere
 - rigida (rotazione + traslazione)
 - affine
 - non-rigida

Registered = $T(\text{Moving})$



Moving



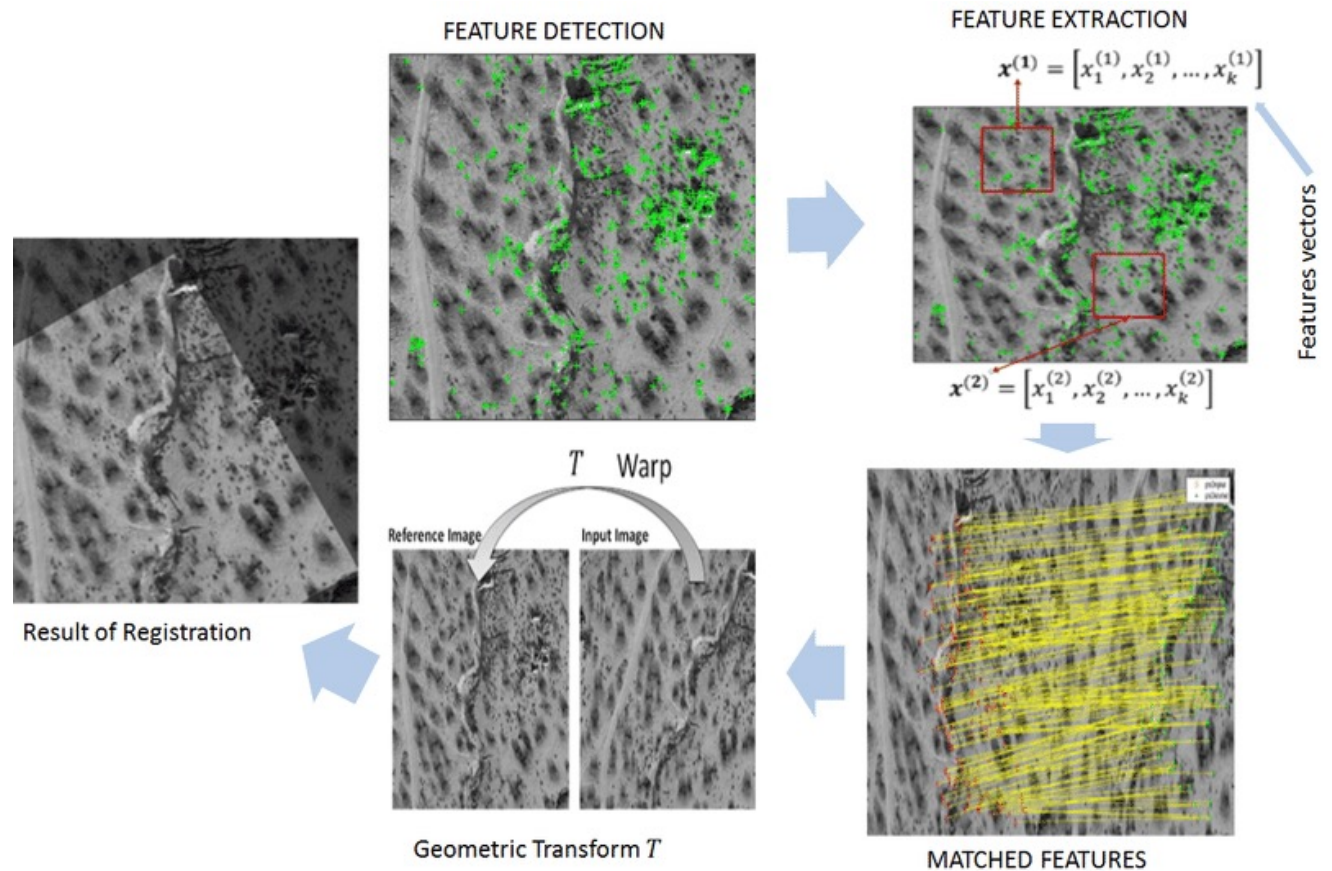
Reference



Sovrapposizione
Registered-Reference

Feature-based Image Registration

- Pipeline:
 - Feature detection
 - Feature description
 - Matching
 - Stima trasformazione T
 - Registra immagini in base a T



Esercitazione...

- Funzioni utili
- OpenCV (cv2)
 - cv2.imread() → caricare immagini
 - cv2.imshow(), cv2.waitKey() → visualizzazione
 - cv2.SIFT_create() / cv2.ORB_create() → feature detection
 - detectAndCompute() → keypoints + descrittori
 - cv2.BFMatcher() → matching descrittori
 - knnMatch() → KNN matching
 - cv2.drawMatches() → visualizzazione match
 - cv2.estimateAffine2D() → stima trasformazione (RANSAC)
 - cv2.warpAffine() → applicare trasformazione
 - cv2.transform() → trasformare punti
 - cv2.circle() → disegnare punti
- Numpy (np)
 - np.array() → gestione dati
 - np.float32() → conversione tipi
 - np.linalg.norm() → distanza euclidea
 - np.mean(), np.sqrt() → statistiche
 - np.random.permutation() → selezione casuale
 - np.genfromtxt() → leggere file (GT)

Multimodal Registration

- Immagini da registrare hanno modalità diverse:
 - RGB vs IR
 - CT vs MRI
- Quindi le feature non corrispondono direttamente
- **Approccio alternativo:** formuliamo un problema di ottimizzazione numerica che ottimizza **un criterio di similarità Sim** tra Immagine Reference I_{ref} e Immagine Registrata (cioè moving image I_{mov} trasformata in base a T)
 - Mutual Information (MI)
 - Cross-correlation
 - Deep learning (recenti)

$$T^* = \max_T Sim(I_{ref}, T(I_{mov}))$$

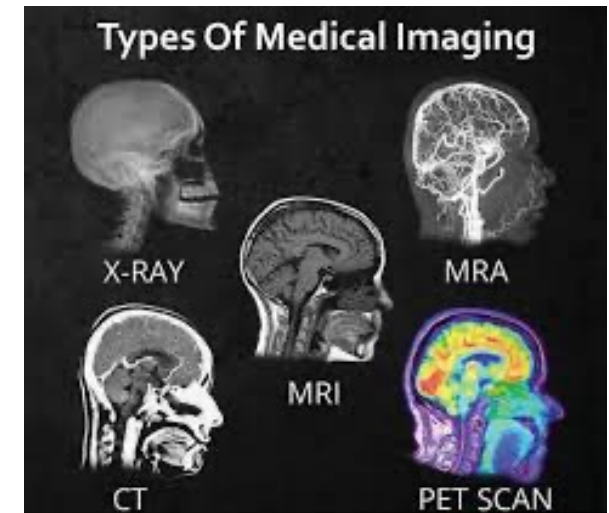


Image Entropy

- Misura l'incertezza/quantità di informazione di un'immagine
- Consideriamo i valori di intensità di una immagine come variabile aleatoria (X)
- Distribuzione dei livelli di intensità $l \rightarrow$ istogramma normalizzato $p(l)$

$$H(I) = - \sum_l p(l) \log p(l)$$

- Entropia alta $\rightarrow$ immagine ricca di dettagli
- Entropia bassa $\rightarrow$ immagine uniforme
- Pipeline:
 - Calcola istogramma dei livelli di intensità dei pixel (es. 256 livelli)
 - Normalizza istogramma
 - Applica la formula dell'entropia

Joint Entropy vs Conditional Entropy

- **Joint Entropy (Entropia Congiunta)** misura l'incertezza complessiva di due immagini X e Y insieme
 - Considera due immagini allineate pixel per pixel
 - Costruisci joint histogram (asse X → intensità immagine X, asse Y → intensità immagine Y)
 - Conta le co-occorrenze e normalizza, ovvero calcola $p(i,j)$
 - Calcola Entropia $H(X,Y) = - \sum_{(i,j)} p(i,j) \log p(i,j)$
 - Alto $H(X,Y)$ → grande variabilità combinata
 - Basso $H(X,Y)$ → relazioni più strutturate tra immagini
- **Conditional Entropy (Entropia Condizionata)** misura quanta incertezza resta in X conoscendo Y
 - $H(X|Y) = H(X,Y) - H(Y)$

Mutua Informazione

- Misura quanta informazione condividono due immagini (X) e (Y)

$$MI(X,Y) = H(X) + H(Y) - H(X,Y)$$

- Non richiede intensità simili (quindi le immagini possono essere multimodali)
- Usa dipendenza statistica tra immagini

Ottimizzare una funzione

- Ottimizzazione con `scipy.optimize`
- Funzione: $f(x) = (x - 3)^2 + 2$
- il minimo è in ($x = 3$)

```
1  import numpy as np
2  from scipy import optimize
3
4  # funzione da minimizzare
5  def f(x):
6      |   return (x - 3)**2 + 2
7
8  # punto iniziale
9  x0 = 0
10
11 # ottimizzazione
12 result = optimize.minimize(f, x0)
13
14 print("Soluzione:", result.x)
15 print("Valore funzione:", result.fun)
```

```
Soluzione: [3.00000003]
Valore funzione: 2.0000000000000001
```

Algoritmi di Ottimizzazione in `scipy.optimize`

- `optimize.minimize` NON usa un solo metodo. Possiamo scegliere diversi algoritmi di ottimizzazione
- `result = optimize.minimize(f, x0, method='BFGS')`
- `BFGS` → veloce, usa gradienti (default comune)
- `Nelder-Mead` → non usa gradienti (più robusto)
- `Powell` → utile per funzioni non lisce
- `CG` → Conjugate Gradient
- La funzione da ottimizzare (es. MI) può essere:
 - non lineare
 - rumorosa
- la scelta del metodo influenza il risultato. Come selezioniamo metodo migliore?

Dataset Split & Hyperparameter Tuning

- La registrazione con Mutual Information richiede scelte di parametri:
 - bin del joint histogram
 - tipo di interpolazione
 - ottimizzazione (step size, iterazioni)
- La **metodologia corretta** per evitare soluzioni “tarate” su uno specifico caso richiede di dividere il dataset in:
 - Validation set → scelta iperparametri
 - Test set → valutazione finale (mai usato prima)
- Su validation set:
 - massimizza ($MI(X, T(Y))$)
 - scegli i parametri migliori facendo esperimenti per diversi valori degli iperparametri
- Fissa gli iper-parametri
 - Valuta su test set:
 - errore di registrazione / MI finale

Assignment 1 – deadline 09/04/2026

- Registrazione multimodale (RGB / IR)
- Stimare trasformazione T che massimizza la Mutua Informazione
- Modello: T è una Roto-traslazione (3 parametri)
- Ottimizzazione: usare `scipy.optimize.minimize`
 - Metodi: Powell, Nelder-Mead, BFGS
 - Inizializzazione: identità
- Warping: usare
 - `cv2.warpAffine`
 - Interpolazione bilineare

Assignment 1 – deadline 09/04/2026

- Dataset: 16 coppie (RGB / IR):
 - 8 validation
 - 8 test
- Protocollo:
 - Validation: scelta iperparametri (bin: 64 / 128 / 256) e metodo di ottimizzazione
 - Test: valutazione finale (NO tuning)
- Metriche:
 - Errore medio:
 - Traslazione
 - rotazione (radianti)
 - Confronto con ground truth
- Consegnare:
 - Codice completo e riproducibile
 - Max 10 slide:
 - metodo, risultati, confronto bin
 - andamento MI durante ottimizzazione
- Output: immagini allineate + differenze

